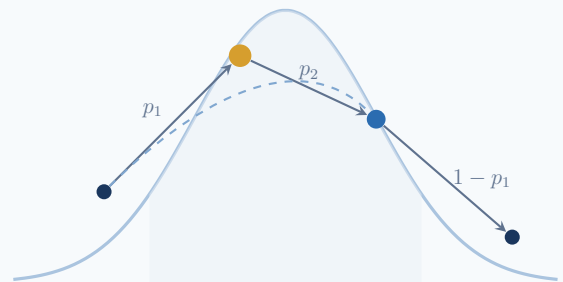


Especialización en ciencia y analítica de datos

Sistemas de aprendizaje automático

Aprendizaje de datos históricos

JAVIER ANDRÉS CAUSIL MARTÍNEZ



Índice general

Prefacio	IX
I Algoritmos y Modelos en Machine Learning	1
1. Minería de datos	3
1.1. ¿Qué áreas relaciona?	3
1.2. ¿Cuál es su objetivo principal?	4
1.3. Organización: El Proceso CRISP-DM	4
2. Conceptos básicos	5
2.1. Algoritmos y modelos	5
2.1.1. 2. Hiperparámetros (Lo que el humano configura)	9
2.1.2. Ejemplo Gráfico: Red Neuronal	9
2.1.3. Árboles de Decisión (Decision Trees)	9
2.1.4. Modelos de Mezcla Gaussiana (GMM)	10
2.2. Parámetros e hiperparámetros	11
2.2.1. 1. En la teoría y metodología (CRISP-DM)	11
2.2.2. 2. En la práctica y el código (Script de Programación)	11
2.3. Gestión de Parámetros e Hiperparámetros en el Flujo de Trabajo	12
2.3.1. 1. Búsqueda en Cuadrícula (Grid Search)	12
2.3.2. 2. Búsqueda Aleatoria (Random Search)	13
2.3.3. 3. Optimización Bayesiana	13
2.3.4. 4. Algoritmos Evolutivos (Algoritmos Genéticos)	14
2.3.5. 5. Hyperband (y Successive Halving)	14
2.3.6. 6. Optimización Basada en Gradientes (Gradient-based Optimization)	14
2.3.7. 7. Inteligencia de Enjambre (Swarm Intelligence - ej. PSO)	15
2.3.8. 8. Búsqueda Manual (El Toque Experto)	15
2.4. Técnicas para la Optimización de Hiperparámetros	16
2.5. Aclaración de Conceptos	16
2.6. El Duelo: Enfoques y Diferencias	16
2.6.1. 1. Programación Tradicional (El enfoque manual)	16
2.6.2. 2. Aprendizaje Automático (El enfoque basado en datos)	16
2.7. Ejemplo de la Vida Real: Filtro Anti-Spam	16
2.8. Representación Gráfica del Paradigma	17
2.9. Aprendizaje Profundo (Deep Learning)	18
2.9.1. 1. La Diferencia Clave: Aprendizaje de Representaciones	18

2.9.2.	2. Estructura Matemática Fundamental	18
2.9.3.	3. Representación Gráfica: ML Clásico vs. Deep Learning	19
2.10.	Etapas del Análisis de Datos	19
2.10.1.	Datos	20
2.10.2.	Preprocesamiento y Limpieza (<i>Data Wrangling</i>)	23
2.10.3.	3. Análisis Exploratorio de Datos (EDA)	23
2.10.4.	4. Modelamiento y Analítica Avanzada	23
2.10.5.	5. Visualización y Comunicación del Valor	23
2.11.	Ecosistema Tecnológico y Perfiles Profesionales	23
2.11.1.	1. El Ecosistema Tecnológico (Los Conjuntos)	24
2.11.2.	2. El Ecosistema de Perfiles (La Cadena de Valor del Dato)	24
2.11.3.	3. Ejemplo Aplicado: Sistema de Inteligencia Turística	25

II Conceptos Fundamentales 27

III Estadística 29

Índice de figuras

1.1.	*	3
2.1.	Visualización del flujo: Cómo los algoritmos utilizan los datos para ajustar parámetros y generar un modelo predictivo.	6
2.2.	Diseño limpio del modelo: Los hiperparámetros dictan la estructura global que se muestra abajo, mientras que el parámetro (peso w) es un valor interno que el algoritmo calcula de forma precisa para la conexión.	9
2.3.	Estructura matemática de un Árbol de Decisión: Un flujo jerárquico de particiones lógicas.	10
2.4.	Representación de un GMM: Distribuciones probabilísticas continuas que se superponen para modelar densidades complejas.	10
2.5.	El cambio fundamental propuesto por el <i>Machine Learning</i> : En lugar de programar las reglas para obtener respuestas, proveemos las respuestas para que la máquina descubra las reglas.	17
2.6.	Diferencia en el flujo de trabajo: El Aprendizaje Profundo elimina la dependencia de la ingeniería manual de características, delegando a la red neuronal tanto la extracción de representaciones como la decisión final.	19
2.7.	Relación de subconjuntos tecnológicos: El Aprendizaje Profundo es una especialización del Machine Learning, que a su vez es una rama de la Inteligencia Artificial.	24

Índice de tablas

2.1. Identificación de Parámetros e Hiperparámetros por Modelo	11
2.2. Comparativa Extendida de Técnicas de Optimización de Hiperparámetros	15
2.3. Clasificación de variables y operaciones principales en la etapa de EDA.	21
2.4. Dimensiones y tareas analíticas de los <i>toy datasets</i> más comunes.	23

Prefacio

Este libro busca ser un compendio de práctica y teoría para entender el aprendizaje automático.

PARTE I

Algoritmos y Modelos en Machine Learning

CAPÍTULO 1

Minería de datos

La minería de datos o data mining, hace referencia a todas las técnicas que permiten extraer conocimiento de un conjunto de datos para descubrir, si es que existen, relaciones, modelos, regularidades o patrones que subyacen en un conjunto de datos y que, a priori, desconocemos.

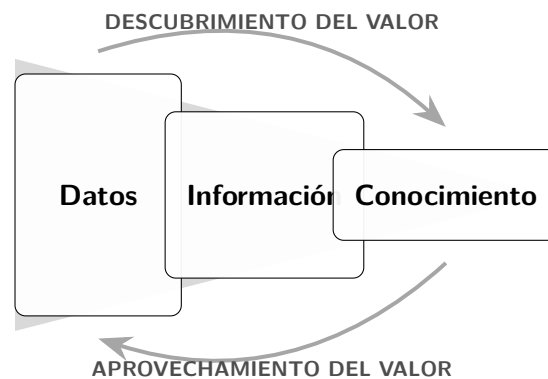


Figura 1.1: *

Figura 1.3. Cadena de valor del dato.

Abarca técnicas de limpieza, transformación y análisis de datos con el fin de descubrir patrones y características específicas. Mientras que el aprendizaje automático engloba los algoritmos y modelos para que las máquinas aprendan y resuelvan tareas automáticamente.

1.1 ¿Qué áreas relaciona?

Cómo es el proceso de analizar grandes volúmenes de información (*Big Data*) para descubrir patrones ocultos, correlaciones, tendencias y anomalías que no son evidentes a simple vista.

Es un campo interdisciplinario que se encuentra justo en la intersección de tres grandes áreas:

- **Estadística:** Para entender la distribución y la probabilidad de los datos.
- **Machine Learning:** Para aplicar los algoritmos que automatizan la búsqueda de patrones.
- **Sistemas de Bases de Datos:** Para almacenar y gestionar eficientemente la información masiva.

1.2 ¿Cuál es su objetivo principal?

El objetivo fundamental de la minería de datos es **extraer conocimiento útil a partir de datos crudos y transformarlo en información accionable**.

En términos prácticos, busca responder preguntas complejas para tomar decisiones estratégicas. Por ejemplo:

- **Predecir el futuro:** ¿Qué clientes tienen más probabilidad de cancelar su suscripción el próximo mes?
- **Clasificar:** ¿Esta transacción bancaria es legítima o es un fraude?
- **Agrupar (Clustering):** ¿Cómo podemos dividir a nuestros compradores en segmentos con gustos similares para lanzar campañas personalizadas?

1.3 Organización: El Proceso CRISP-DM

Para que la minería de datos se ejecute de forma estructurada, la industria utiliza metodologías estándar. La más famosa y utilizada a nivel mundial es **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*). Este marco de trabajo organiza todo proyecto en 6 fases cíclicas:

1. **Comprensión del Negocio (Business Understanding):** Antes de procesar datos, se debe entender el problema a resolver. Se definen los objetivos comerciales y los criterios de éxito del proyecto.
2. **Comprensión de los Datos (Data Understanding):** Se recolectan los datos iniciales y se realiza una exploración preliminar. Se identifican problemas de calidad (datos faltantes, errores) y se descubren pistas iniciales.
3. **Preparación de los Datos (Data Preparation):** Suele ser la fase que más tiempo consume (70 % - 80 % del proyecto). Involucra limpiar bases de datos, transformar variables, unir tablas y formatear la información para los algoritmos.
4. **Modelado (Modeling):** Se seleccionan y aplican diversas técnicas de Machine Learning (como las vistas anteriormente). Se calibran los parámetros (hiperparámetros) para encontrar el modelo con mejor rendimiento.
5. **Evaluación (Evaluation):** Se analiza el modelo rigurosamente. No solo se verifica su precisión matemática, sino que se valida si realmente cumple con el objetivo de negocio planteado en la primera fase.
6. **Despliegue (Deployment):** El modelo se lleva al entorno real. Puede implementarse como un reporte técnico, un panel interactivo (*dashboard*) o integrarse como un sistema automatizado.

CAPÍTULO 2

Conceptos básicos

En este capítulo tratamos de entender el flujo de los algoritmos y los modelos que se generan.

2.1 Algoritmos y modelos

¿Cuál es la diferencia principal? A menudo se usan como sinónimos, pero en la práctica son dos conceptos distintos que trabajan juntos:

- **El Algoritmo (El Motor / La Receta):** Es el procedimiento matemático o el conjunto de reglas que la computadora utiliza para analizar los datos, encontrar patrones y “aprender” de ellos. Ejemplos de algoritmos son el Descenso del Gradiente o la Retropropagación (*Backpropagation*).
- **El Modelo (El Resultado / El Plato Terminado):** Es la representación matemática final que resulta **después** de que el algoritmo ha procesado los datos de entrenamiento. Es esencialmente una función matemática a la que le entregas nuevos datos de entrada para obtener una predicción o clasificación de salida.

En resumen: **Algoritmo + Datos de Entrenamiento = Modelo de Machine Learning.**
Explicación del flujo de Machine Learning: Este diagrama ilustra la transición desde la información sin procesar hasta la toma de decisiones automatizada:

1. El punto de partida:

El proceso arranca cuando los **Datos de Entrada** alimentan al **Algoritmo**. El algoritmo actúa como un motor de optimización que busca patrones matemáticos. Para que este motor funcione adecuadamente, el científico de datos establece reglas previas llamadas *Hiperparámetros* (configuraciones manuales que guían el aprendizaje).

2. La consolidación (El Aprendizaje):

Al procesar los datos, el algoritmo ajusta sus valores internos. El resultado final de esta fase es el **Modelo**. Este modelo almacena los *Parámetros Aprendidos* (como los pesos y sesgos), que son los valores exactos que el algoritmo descubrió por sí solo para resolver el problema.

3. La fase de inferencia:

Una vez que la etapa de aprendizaje concluye, el algoritmo pasa a un segundo plano. Ahora es el **Modelo**, con sus parámetros ya calibrados, el que recibe datos nuevos que nunca antes ha visto para aplicar su función matemática y generar una **Predicción** final.

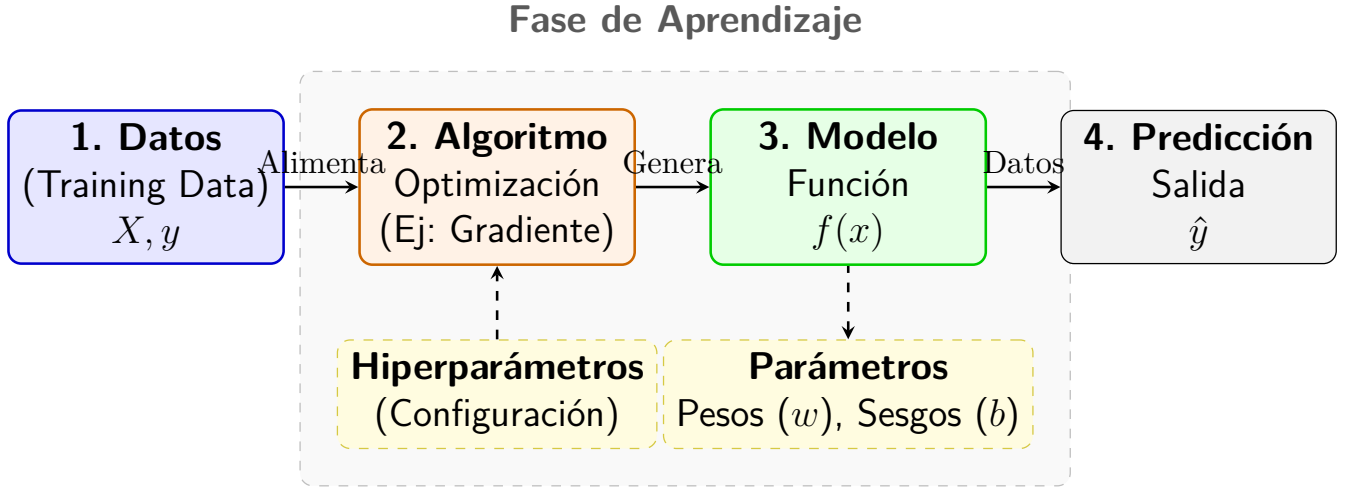


Figura 2.1: Visualización del flujo: Cómo los algoritmos utilizan los datos para ajustar parámetros y generar un modelo predictivo.

Ejemplos de Algoritmos y sus Modelos

A continuación, se presentan algunos de los enfoques más clásicos en Machine Learning, cómo funciona su algoritmo y la estructura matemática del modelo resultante.

1. **Regresión Lineal (Linear Regression):** Se utiliza para predecir un valor numérico continuo (por ejemplo, el precio de una casa basado en su tamaño).

- **El Algoritmo:** Usualmente se emplea el método de *Mínimos Cuadrados Ordinarios (OLS)* o el *Descenso del Gradiente* para encontrar los coeficientes que minimicen el error cuadrático entre la predicción y los datos reales.
- **El Modelo:** Una vez entrenado, el modelo es una ecuación lineal (una línea recta o un hiperplano) donde se han calculado los pesos ideales (β).

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n \quad (2.1)$$

(Donde y es la predicción, x_i son las características de entrada, β_0 es el sesgo y β_i son los pesos aprendidos).

2. **Regresión Logística (Logistic Regression)** A pesar de su nombre, se utiliza para **clasificación** binaria (por ejemplo, decidir si un correo es “Spam” o “No Spam”).

- **El Algoritmo:** Utiliza la *Maximización de la Verosimilitud* (Maximum Likelihood Estimation) para ajustar los pesos del modelo a los datos de entrenamiento.
- **El Modelo:** Toma una ecuación lineal y la pasa por una función Sigmoide para comprimir el resultado y convertirlo en una probabilidad matemática entre 0 y 1.

$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}} \quad (2.2)$$

(Donde $P(y = 1|x)$ es la probabilidad de que la entrada x pertenezca a la clase 1).

3. **Máquinas de Vectores de Soporte (SVM)** Se utiliza principalmente para clasificación compleja. Busca trazar la mejor “frontera” posible entre diferentes tipos de datos.

- **El Algoritmo:** Es un solucionador de optimización matemática que busca maximizar el “margen” (la distancia) entre los puntos de datos más cercanos de diferentes clases (los vectores de soporte). El objetivo del algoritmo es minimizar $\frac{1}{2}||w||^2$.
- **El Modelo:** El resultado es la ecuación del **hiperplano óptimo** que separa las clases en el espacio multidimensional.

$$w^T x + b = 0 \quad (2.3)$$

(Donde w es el vector de pesos que determina la orientación del hiperplano, x es la entrada y b es el sesgo).

4. **Redes Neuronales Artificiales (ANN)** Es la base del *Deep Learning*. Modela relaciones altamente complejas y no lineales.

- **El Algoritmo:** Utiliza *Backpropagation* (Propagación hacia atrás) junto con un optimizador para ajustar millones de pesos en la red calculando derivadas parciales (gradientes) de la función de pérdida.
- **El Modelo:** Es una red de funciones matemáticas interconectadas. Para una sola “neurona” dentro de la red, el modelo matemático calcula una suma ponderada y la pasa por una función de activación no lineal σ (como ReLU o Sigmoide).

$$a = \sigma \left(\sum_{i=1}^n w_i x_i + b \right) \quad (2.4)$$

(Donde a es la salida o activación de la neurona, w_i son los pesos, x_i son las entradas y b es el sesgo).

5. **Clasificador Naïve Bayes (Naïve Bayes)** Este es un modelo generativo fundamentado en el Teorema de Bayes, utilizado ampliamente en clasificación de textos y análisis de sentimientos. Se le llama “ingenuo” (*naïve*) porque asume independencia condicional estricta entre las características dado la clase.

- **El Algoritmo:** Utiliza la estimación de Máxima Probabilidad a Posteriori (MAP) contando frecuencias en los datos de entrenamiento para calcular empíricamente la probabilidad a priori de cada clase $P(y)$ y las probabilidades condicionales de cada característica $P(x_i|y)$.
- **El Modelo:** Es una regla de decisión matemática. Para una nueva observación con características $x = (x_1, \dots, x_n)$, el modelo asigna la clase $\hat{y}$ que maximiza la probabilidad conjunta (gracias al supuesto de independencia):

$$\hat{y} = \arg \max_y P(y) \prod_{i=1}^n P(x_i | y) \quad (2.5)$$

6. **Agrupamiento K-Medias (K-Means Clustering)** Es el algoritmo por excelencia para el aprendizaje no supervisado. Su objetivo es particionar un conjunto de n observaciones en k grupos (clústeres), donde cada observación pertenece al grupo cuyo valor medio sea más cercano.

- **El Algoritmo:** Típicamente emplea el *Algoritmo de Lloyd* (un enfoque heurístico iterativo de Expectativa-Maximización). En cada iteración, reasigna los puntos al centroide más cercano y luego recalcula la posición del centroide como el centro de masa geométrico del grupo. El objetivo del algoritmo es minimizar la varianza intra-clúster (la suma de los errores cuadráticos):

$$J = \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - \mu_j\|^2 \quad (2.6)$$

- **El Modelo:** Una vez que converge, el modelo no es una ecuación continua, sino un conjunto discreto de k vectores en $\mathbb{R}^d$ que representan los centroides finales $\{\mu_1, \mu_2, \dots, \mu_k\}$. Para predecir a qué grupo pertenece un nuevo punto x , el modelo evalúa la métrica de distancia:

$$\text{Clúster}(x) = \arg \min_{j \in \{1, \dots, k\}} \|x - \mu_j\|^2 \quad (2.7)$$

7. **Análisis de Componentes Principales (PCA)** Aunque a menudo se clasifica como una técnica de preprocesamiento, es un modelo matemático de aprendizaje no supervisado utilizado para la reducción de dimensionalidad, preservando la mayor cantidad de varianza posible.

- **El Algoritmo:** Implica álgebra lineal pura. Toma la matriz de datos centrados X , calcula su matriz de covarianza $\Sigma = \frac{1}{n-1} X^T X$ y luego realiza una descomposición espectral para encontrar sus autovalores (λ) y autovectores (v). Se seleccionan los k autovectores correspondientes a los k autovalores más grandes.

$$\Sigma v = \lambda v \quad (2.8)$$

- **El Modelo:** Es una matriz de proyección ortogonal $W_{d \times k}$ (cuyas columnas son los k autovectores principales). Al recibir un nuevo dato original x (de dimensión d), el modelo lo transforma a un nuevo espacio latente z (de dimensión k):

$$z = xW \quad (2.9)$$

En el Aprendizaje Automático (Machine Learning), es vital distinguir entre las variables que el modelo aprende por sí solo y las configuraciones que el científico de datos debe establecer manualmente.

1. Parámetros (Lo que la máquina aprende)

Son las variables internas del modelo cuyos valores se ajustan automáticamente a partir de los datos durante el proceso de entrenamiento. El programador no los define directamente.

- **Origen:** Se estiman y optimizan mediante el algoritmo (ej. Descenso del Gradiente).
- **Ejemplos:** Los coeficientes (β) en una regresión lineal, los vectores de soporte en SVM, o los pesos (w) y sesgos (b) en una red neuronal.

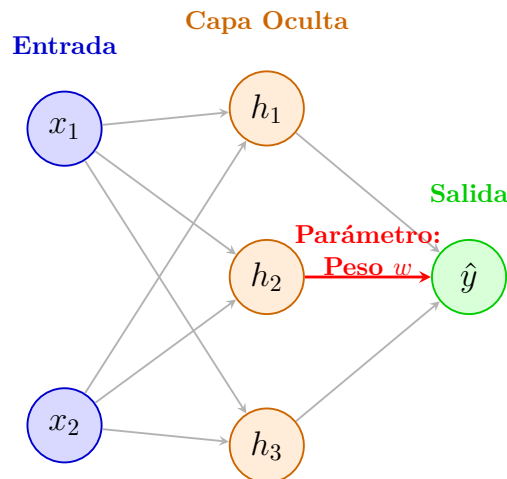
2.1.1 2. Hiperparámetros (Lo que el humano configura)

Son las variables de configuración externas que definen la “arquitectura” del modelo o cómo se comportará el algoritmo de aprendizaje. Deben fijarse **antes** de que comience el entrenamiento.

- **Origen:** Los define empíricamente el científico de datos o mediante técnicas de búsqueda (Grid Search, Random Search).
- **Ejemplos:** La tasa de aprendizaje (*learning rate*), el número de capas ocultas en una red neuronal, el valor de K en K-Medias, o la profundidad máxima de un árbol de decisión.

2.1.2 Ejemplo Gráfico: Red Neuronal

En el siguiente gráfico se ilustra una red neuronal sencilla para visualizar visualmente dónde reside un hiperparámetro y dónde reside un parámetro.



Hiperparámetros (Configuración estructural definida previamente):

- Número de capas ocultas = 1
- Neuronas por capa = 3
- Tasa de aprendizaje (α) = 0.01

Figura 2.2: Diseño limpio del modelo: Los hiperparámetros dictan la estructura global que se muestra abajo, mientras que el parámetro (peso w) es un valor interno que el algoritmo calcula de forma precisa para la conexión.

2.1.3 Árboles de Decisión (Decision Trees)

Este modelo segmenta el espacio de características mediante reglas de decisión condicionales, estructuradas de forma jerárquica.

- **El Algoritmo:** Evalúa de forma iterativa todas las variables para encontrar el punto de corte óptimo que maximice la homogeneidad de los subgrupos resultantes, utilizando métricas como la Ganancia de Información o el Índice Gini.

- **El Modelo:** Es una estructura en árbol donde los nodos internos representan preguntas lógicas y las hojas contienen las predicciones finales.

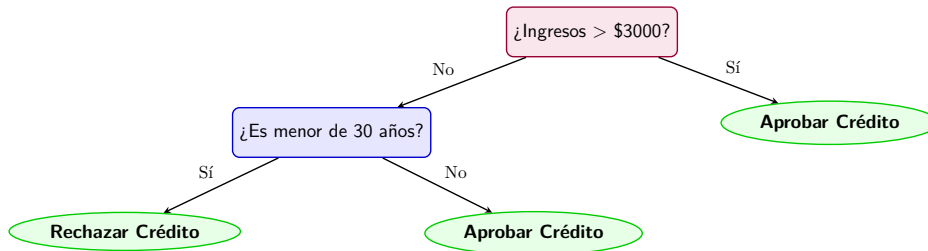


Figura 2.3: Estructura matemática de un Árbol de Decisión: Un flujo jerárquico de particiones lógicas.

2.1.4 Modelos de Mezcla Gaussiana (GMM)

Es un modelo de agrupamiento probabilístico que asume que los datos provienen de una combinación lineal de múltiples distribuciones normales independientes.

- **El Algoritmo:** Utiliza el método iterativo de **Expectación-Maximización (EM)** para aproximar de forma óptima la ubicación y dispersión de las densidades probabilísticas.
- **El Modelo:** Matemáticamente se expresa como una densidad de probabilidad conjunta:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k) \quad (2.10)$$

(Donde π_k representa el peso o importancia de la mezcla, μ_k el centro o media, y Σ_k la matriz de covarianza que define la forma geométrica del grupo k).

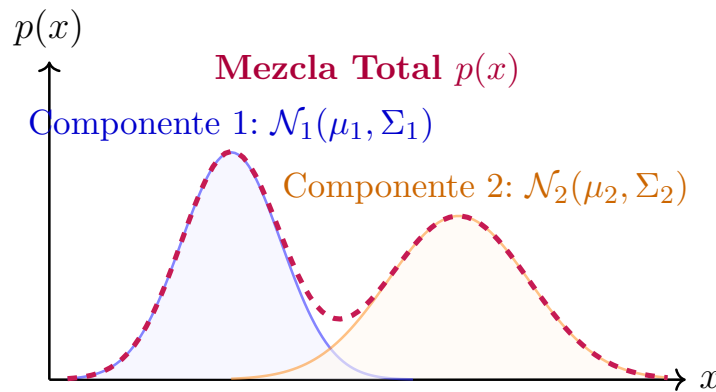


Figura 2.4: Representación de un GMM: Distribuciones probabilísticas continuas que se superponen para modelar densidades complejas.

Tabla 2.1: Identificación de Parámetros e Hiperparámetros por Modelo

Modelo	Hiperparámetros (Tus decisiones previas)	Parámetros (Lo que el modelo aprendió)
Red Neuronal	Número de capas, neuronas por capa, tasa de aprendizaje (α).	Pesos (w) de las conexiones y sesgos (b).
Árbol de Decisión	Profundidad máxima del árbol, criterio de división (Gini/Entropía).	Variables seleccionadas para los nodos y umbrales de corte (ej: \$3000).
GMM	Número de componentes o grupos a buscar (K).	Medias (μ), covarianzas (Σ) y pesos de cada campana (π).

2.2 Parámetros e hiperparámetros

En el flujo de trabajo de *Machine Learning* y Ciencia de Datos, los parámetros e hiperparámetros se explican, se definen y se gestionan en dos momentos y secciones completamente distintas del proyecto.

A continuación, se detalla exactamente dónde se encuentra cada uno de ellos tanto en la teoría (metodologías como CRISP-DM) como en la práctica (el código de programación).

2.2.1 1. En la teoría y metodología (CRISP-DM)

Si la documentación o el reporte técnico se organiza bajo el marco de la minería de datos, ambos conceptos viven en la **Fase 4: Modelado**, pero se explican en subsecciones separadas:

Los Hiperparámetros se explican en el “Diseño de la Prueba” (Configuración):

Al inicio de la fase de modelado, se abre una sección para justificar la arquitectura. Aquí se explican qué valores fueron elegidos para el modelo **antes** de entrenar.

Ejemplo: “Se seleccionó un modelo de Red Neuronal con 1 capa oculta de 3 neuronas y una tasa de aprendizaje $\alpha = 0,01$ debido a restricciones de cómputo”.

Los Parámetros se explican en la “Evaluación del Modelo” (Resultados):

Una vez que el algoritmo termina de entrenar, se genera una sección de resultados donde se muestran las variables que el modelo descubrió por sí mismo a través de la optimización.

Ejemplo: “El modelo lineal final obtuvo un sesgo $\beta_0 = 2,5$ y un coeficiente de peso $\beta_1 = 0,8$, lo que demuestra una relación positiva entre las variables”.

2.2.2 2. En la práctica y el código (Script de Programación)

Al observar un código real (utilizando librerías como *Scikit-Learn* o *TensorFlow* en Python), la distinción de la ubicación de ambos conceptos es inmediata:

A. Los Hiperparámetros están en la Instanciación (Definición)

Se especifican y se escriben en la línea de código donde **se crea el objeto del modelo**. Son los argumentos de configuración que el programador pasa explícitamente entre los paréntesis.

```
# AQUÍ CONFIGURAS LOS HIPERPARÁMETROS (Manual)
modelo = MLPClassifier(hidden_layer_sizes=(3,), learning_rate_init=0.01)
```

B. Los Parámetros nacen en el Entrenamiento (Función .fit())

No existen al principio del script. El algoritmo los calcula internamente en la línea de código donde el modelo procesa la matriz de datos. Tras ejecutar esa instrucción, el modelo almacena los parámetros aprendidos de forma interna.

```
# El algoritmo procesa los datos y calcula los parámetros automáticamente
modelo.fit(X_train, y_train)

# AQUÍ REVISAS LOS PARÁMETROS RESULTANTES (Automáticos)
print(modelo.coefs_) # Muestra los pesos calculados por la máquina
```

Resumen para el documento

Para estructurar esto de forma impecable en la sección de **Algoritmos y Modelos**:

- Utiliza la sección de **Configuración/Arquitectura** para justificar los *hiperparámetros* (las decisiones iniciales tomadas por el científico de datos).
- Utiliza la sección de **Resultados/Ecuación Final** para exponer los *parámetros* obtenidos (los valores que el algoritmo dedujo tras procesar la información).

2.3 Gestión de Parámetros e Hiperparámetros en el Flujo de Trabajo

Encontrar los hiperparámetros correctos (proceso conocido como *Hyperparameter Tuning*) suele ser uno de los mayores desafíos en la Ciencia de Datos. Al no existir una fórmula matemática directa que dé la respuesta exacta desde el inicio, es un proceso empírico de prueba y error, pero que debe ejecutarse de manera estructurada.

A continuación, se presentan las cuatro técnicas principales utilizadas en la industria para automatizar y optimizar esta búsqueda.

2.3.1 1. Búsqueda en Cuadrícula (Grid Search)

Es la técnica más tradicional y exhaustiva, conocida informalmente como “fuerza bruta”.

- **Cómo funciona:** Se define un conjunto discreto de valores para cada hiperparámetro. El algoritmo crea una cuadrícula y prueba **absolutamente todas** las combinaciones posibles, entrenando un modelo para cada una y evaluando su rendimiento.

- **Cuándo usarla:** Cuando se tienen muy pocos hiperparámetros (2 o 3) y el modelo es de entrenamiento rápido.
- **Ventaja:** Garantiza encontrar la mejor combinación exacta dentro de la lista de valores proporcionada.
- **Desventaja:** Es computacionalmente muy costosa. Si se tienen 4 hiperparámetros y se prueban 5 opciones para cada uno, la computadora tendrá que entrenar $5^4 = 625$ modelos distintos.

2.3.2 2. Búsqueda Aleatoria (Random Search)

Es la alternativa inteligente y práctica a la lentitud del *Grid Search*.

- **Cómo funciona:** En lugar de probar todas las combinaciones metódicamente, se define un rango o distribución estadística para cada hiperparámetro. El algoritmo elige valores al azar dentro de esos rangos durante un número de intentos limitado definido por el usuario.
- **Cuándo usarla:** Es el estándar de oro inicial para la mayoría de los proyectos de Machine Learning en la actualidad.
- **Ventaja:** Suele encontrar configuraciones iguales o mejores que *Grid Search* en una fracción del tiempo, ya que no pierde iteraciones probando combinaciones inútiles.
- **Desventaja:** Al ser un proceso estocástico (basado en el azar), no garantiza encontrar el óptimo global absoluto.

2.3.3 3. Optimización Bayesiana

Es el estándar moderno y la técnica más sofisticada, especialmente utilizada en *Deep Learning* y modelos computacionalmente pesados.

- **Cómo funciona:** Utiliza principios de probabilidad condicional (Teorema de Bayes) para “aprender” de las combinaciones anteriores. Construye un modelo probabilístico interno para estimar qué zonas del espacio de hiperparámetros tienen más probabilidades de éxito, guiando las siguientes iteraciones hacia allá.
- **Cuándo usarla:** Cuando entrenar el modelo toma horas o días, y no es viable desperdiciar tiempo en combinaciones ineficientes.
- **Ventaja:** Es extremadamente eficiente. Encuentra los mejores resultados entrenando la menor cantidad de modelos posibles.
- **Desventaja:** Su formulación matemática es compleja, aunque en la práctica se soluciona utilizando librerías especializadas en Python como Optuna o Hyperopt.

2.3.4 4. Algoritmos Evolutivos (Algoritmos Genéticos)

Es un enfoque heurístico inspirado en la teoría de la evolución biológica.

- **Cómo funciona:** Se crea una “población” inicial de hiperparámetros aleatorios. Tras entrenar y evaluar los modelos, las configuraciones con mejores métricas “sobreviven”. Estas configuraciones ganadoras se combinan entre sí (cruce) y sufren pequeñas variaciones (mutación) para generar una nueva generación de pruebas.
- **Cuándo usarla:** En arquitecturas de redes neuronales altamente complejas o espacios de búsqueda gigantescos.
- **Ventaja:** Excelente para escapar de óptimos locales (cuando el modelo se estanca creyendo haber encontrado la mejor solución, pero existe una mejor en otra región del espacio).
- **Desventaja:** Requiere una gran potencia de cálculo, ya que se deben procesar múltiples generaciones completas antes de converger.

2.3.5 5. Hyperband (y Successive Halving)

Esta es una de las técnicas modernas más populares para ahorrar tiempo y recursos de computación, especialmente en la nube y en sistemas de *AutoML* (Machine Learning Automatizado).

- **La idea:** Combina la Búsqueda Aleatoria con un mecanismo de “supervivencia del más apto” basado en presupuestos (por ejemplo, tiempo máximo o número de iteraciones).
- **Cómo funciona:** Inicia entrenando muchas configuraciones al azar, pero solo por un periodo muy corto. Evalúa cuáles van por buen camino y detiene prematuramente (*Early Stopping*) a los modelos con bajo rendimiento. Los recursos computacionales liberados se reasignan exclusivamente a los modelos prometedores para que continúen su entrenamiento.

2.3.6 6. Optimización Basada en Gradientes (Gradient-based Optimization)

Es un enfoque puramente matemático y altamente especializado.

- **La idea:** Si utilizamos el Descenso del Gradiente para encontrar los parámetros internos de una red neuronal, se puede usar un principio similar para encontrar los hiperparámetros.
- **Cómo funciona:** Calcula “hipergradientes” que indican al algoritmo en qué dirección matemática debe mover los hiperparámetros para minimizar el error.
- **Limitación:** Solo es aplicable para hiperparámetros que representan variables continuas (como la tasa de aprendizaje); no funciona para decisiones discretas (como el número de capas).

2.3.7 7. Inteligencia de Enjambre (Swarm Intelligence - ej. PSO)

Es otra rama de la computación evolutiva, estrechamente relacionada con los Algoritmos Genéticos.

- **La idea:** En lugar de imitar la evolución genética, modela el comportamiento social de una bandada de pájaros o un banco de peces buscando alimento (*Particle Swarm Optimization*).
- **Cómo funciona:** Cada “pájaro” (una configuración de hiperparámetros) explora el espacio de búsqueda. Si uno encuentra una región prometedora, informa al resto del enjambre, y todos ajustan su trayectoria matemáticamente para explorar cerca de ese óptimo local.

2.3.8 8. Búsqueda Manual (El Toque Experto)

En el entorno académico, a menudo se le conoce coloquialmente como *Grad Student Descent* (Descenso del Estudiante de Posgrado).

- **Cómo funciona:** Un científico de datos con profunda experiencia en el dominio del problema intuye por dónde empezar a probar configuraciones, ajustando los valores a mano. Aunque carece de automatización, la intuición humana guiada por la experiencia previa a menudo converge en un buen modelo más rápido que una búsqueda ciega.

Tabla 2.2: Comparativa Extendida de Técnicas de Optimización de Hiperparámetros

Técnica	Eficiencia Computacional	Calidad del Resultado	Característica Principal
Grid Search	Muy Baja (Lenta)	Alta	Exhaustiva, fuerza bruta.
Random Search	Media a Alta	Muy Buena	Estocástica, rápida implementación.
Optimiz. Bayesiana	Muy Alta (Rápida)	Excelente	Aprende probabilísticamente del pasado.
Algoritmos Genéticos	Baja a Media	Excelente	Evoluciona generaciones, mutaciones.
Hyperband	Muy Alta (Ahorra recursos)	Muy Buena a Excelente	<i>Early stopping</i> de modelos malos.
Basada en Gradientes	Alta (Solo continuos)	Excelente	Uso de derivadas (hipergradientes).
Intel. de Enjambre	Media	Excelente	Optimización colaborativa (PSO).
Búsqueda Manual	Variable	Variable	Basada en intuición y experiencia.

2.4 Técnicas para la Optimización de Hiperparámetros

2.5 Aclaración de Conceptos

En la literatura técnica en español, los términos **Aprendizaje de Máquina** (traducción literal) y **Aprendizaje Automático** (traducción formal) se refieren exactamente a la misma disciplina: *Machine Learning*.

Para entender su impacto, el verdadero contraste analítico debe hacerse frente a la automatización clásica, es decir, la **Programación Tradicional**.

2.6 El Duelo: Enfoques y Diferencias

La diferencia fundamental entre ambos enfoques radica en **cómo se construyen las reglas** para resolver un problema.

2.6.1 1. Programación Tradicional (El enfoque manual)

En la informática clásica, un humano debe entender el problema a la perfección, deducir la lógica matemática o condicional y escribir las reglas explícitas en código.

- **La Ecuación:** Datos + Reglas dadas por el humano = Respuestas.
- **Limitación:** Si el problema es demasiado complejo o tiene millones de excepciones, es humanamente imposible escribir todas las reglas condicionales (*if/else*).

2.6.2 2. Aprendizaje Automático (El enfoque basado en datos)

En el *Machine Learning*, el humano no programa las reglas lógicas. En su lugar, el humano le entrega a la computadora un conjunto masivo de ejemplos (datos y las respuestas correctas de esos datos), y el algoritmo se encarga de descubrir cuáles son las reglas matemáticas que los conectan.

- **La Ecuación:** Datos + Respuestas históricas = Reglas (El Modelo).
- **Ventaja:** Excelente para problemas donde las reglas son ocultas, dinámicas o demasiado complejas (ej. reconocimiento de voz o visión por computadora).

2.7 Ejemplo de la Vida Real: Filtro Anti-Spam

Supongamos que queremos crear un sistema que detecte si un correo electrónico es *Spam* o no.

Solución con Programación Tradicional: El programador debe escribir reglas estrictas basadas en su intuición:

SI correo contiene "Gana dinero rápido" ENTONCES Spam = Verdadero

SI remitente es desconocido Y contiene enlace ENTONCES Spam = Verdadero

Problema: Los spammers cambian la frase a “Gana dinero rápido” y el sistema tradicional se rompe, obligando al humano a reescribir el código constantemente.

Solución con Aprendizaje Automático: El programador **no** escribe reglas de palabras. Recolecta 10,000 correos marcados como Spam y 10,000 correos normales. Le pasa todo eso al algoritmo. El modelo analiza la frecuencia de palabras, la estructura y metadatos, y construye su propia red de probabilidades. Si los spammers cambian de táctica, solo hay que reentrenar el modelo con los correos nuevos para que aprenda las nuevas reglas solo.

2.8 Representación Gráfica del Paradigma

El siguiente gráfico ilustra cómo se invierten las entradas y salidas entre ambos enfoques.

1. Programación Tradicional



2. Aprendizaje Automático (ML)



Figura 2.5: El cambio fundamental propuesto por el *Machine Learning*: En lugar de programar las reglas para obtener respuestas, proveemos las respuestas para que la máquina descubra las reglas.

2.9 Aprendizaje Profundo (Deep Learning)

El Aprendizaje Profundo (*Deep Learning*) es un subcampo especializado del Aprendizaje Automático, fundamentado en el uso de Redes Neuronales Artificiales (ANN) con múltiples capas ocultas.

Mientras que los algoritmos clásicos de *Machine Learning* alcanzan una asíntota de rendimiento independientemente de la cantidad de datos que se les proporcione, los modelos de aprendizaje profundo tienen la capacidad de continuar mejorando su precisión y escalabilidad a medida que aumenta el volumen de información y la capacidad de cómputo.

2.9.1 1. La Diferencia Clave: Aprendizaje de Representaciones

En el *Machine Learning* tradicional, el rendimiento del modelo depende en gran medida de la **Ingeniería de Características** (*Feature Engineering*). Un experto humano debe extraer manualmente las variables matemáticas o estadísticas más relevantes de los datos crudos antes de pasarlos al algoritmo clasificador.

El Aprendizaje Profundo elimina este cuello de botella mediante el **Aprendizaje de Representaciones** (*Representation Learning*). La red neuronal procesa los datos crudos (como matrices de píxeles en una imagen o secuencias temporales) y descubre automáticamente las características más relevantes de forma jerárquica a través de sus capas.

2.9.2 2. Estructura Matemática Fundamental

Una Red Neuronal Profunda (DNN) puede modelarse como una composición de funciones matemáticas no lineales. Para una capa arbitraria l en una red de L capas, la transformación afín aplicada a la matriz de activaciones de la capa anterior $A^{[l-1]}$, seguida de una función de activación, se define rigurosamente de la siguiente manera:

$$Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]} \quad (2.11)$$

$$A^{[l]} = \sigma^{[l]}(Z^{[l]}) \quad (2.12)$$

Donde:

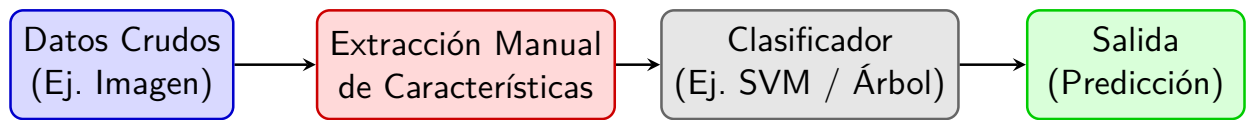
- $W^{[l]}$ es la matriz de pesos de dimensión $(n^{[l]}, n^{[l-1]})$.
- $b^{[l]}$ es el vector de sesgos (*bias*) de dimensión $(n^{[l]}, 1)$.
- $Z^{[l]}$ es la transformación lineal previa a la activación.
- $\sigma^{[l]}$ es la función de activación no lineal, dependiente y diferenciable (e.g., ReLU, Sigmoid, o Tangente Hiperbólica).

El modelo predictivo final es la composición completa de estas L transformaciones matriciales sucesivas, evaluada desde la entrada original X (que equivale a $A^{[0]}$) hasta generar la matriz de salida final $\hat{Y} = A^{[L]}$.

2.9.3 3. Representación Gráfica: ML Clásico vs. Deep Learning

El siguiente esquema ilustra cómo el Aprendizaje Profundo unifica las etapas de representación y clasificación en un único flujo de optimización continua (entrenamiento *end-to-end*).

1. Machine Learning Clásico



2. Deep Learning

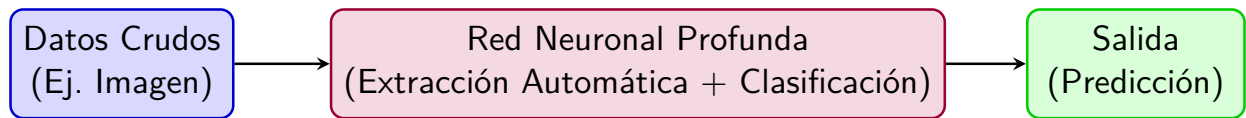


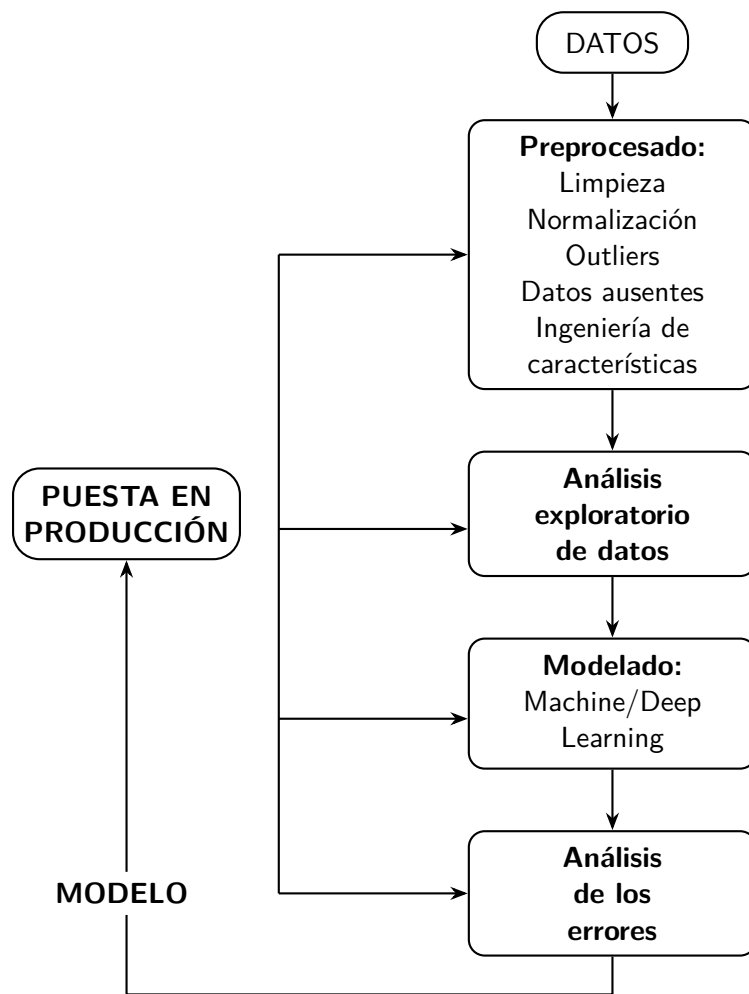
Figura 2.6: Diferencia en el flujo de trabajo: El Aprendizaje Profundo elimina la dependencia de la ingeniería manual de características, delegando a la red neuronal tanto la extracción de representaciones como la decisión final.

2.10 Etapas del Análisis de Datos

Esta sección esta desarrollada basado en [1]. Recursos para profundizar o tener un roadmap es [data-analyst](#).

El proceso de análisis de datos no es un conjunto de tareas aisladas, sino un ciclo iterativo y estructurado que transforma datos crudos en conocimiento accionable y valor estratégico. Lo primero es identificar las variables y patrones contenidos en nuestros datos, si tenemos la información de los datos en una tabla, cada fila sería un patrón y cada columna un variable.

A continuación, se describen las etapas fundamentales que componen este flujo de trabajo.



2.10.1 Datos

Conocer nuestros datos es el punto de partida de cualquier problema basado en datos. En este paso clasificamos nuestras variables entre categóricas o continuas por el tratamiento posterior que les vamos a dar.

Clasificación de Variables en el Análisis de Datos

En el análisis de datos y modelado estadístico, las variables del conjunto de datos se clasifican según su naturaleza matemática en dos grandes familias: **cualitativas** y **cuantitativas**.

1. Variables Cualitativas o Categóricas

Expresan atributos, cualidades o características no numéricas.

Nominales:

Categorías que **no poseen un orden natural o jerárquico**. Solo sirven para etiquetar o clasificar grupos.

- *Ejemplos:* Estado de una reserva (Confirmada, Cancelada), Tipo de alojamiento (Apartamento, Habitación), Método de pago.

Ordinales:

Categorías que **sí poseen un orden o jerarquía interna**, aunque la distancia exacta entre los valores no sea medible.

- *Ejemplos:* Nivel de satisfacción (Bajo, Medio, Alto), Nivel educativo, Clasificación de riesgo.

2. Variables Cuantitativas o Numéricas

Son aquellas que se expresan mediante valores numéricos y permiten realizar operaciones matemáticas.

Discretas:

Surgen del proceso de **contar**. Toman valores enteros y aislados dentro de un rango; no admiten valores intermedios.

- *Ejemplos:* Número de noches de estadía, Cantidad de pasajeros en un vuelo, Total de habitaciones.

Continuas:

Surgen del proceso de **medir**. Pueden tomar cualquier valor real con decimales dentro de un intervalo continuo.

- *Ejemplos:* Precio o tarifa por noche, Tiempo de espera en minutos, Distancia geográfica.

Mapeo en Dataframes (Resumen Estadístico-Técnico)

A continuación se presenta el mapeo típico de estos conceptos estadísticos a tipos de datos lógicos utilizados en entornos de desarrollo (como Python o SQL):

Tabla 2.3: Clasificación de variables y operaciones principales en la etapa de EDA.

Tipo Estadístico	Tipo de Dato (Python/SQL)	Operación Típica en EDA
Nominal	object, string, category	Conteo de frecuencias, Moda, Gráficos de barras.
Ordinal	category (con orden)	Medianas, Percentiles, Gráficos ordenados.
Discreta	int, integer	Sumas, Conteos, Histogramas discretos.
Continua	float, decimal	Promedios, Desviación estándar, <i>Boxplots</i> .

Conjuntos de Datos Triviales en el Análisis de Datos

Un conjunto de datos trivial (*toy dataset*) es una colección de datos de dimensiones reducidas, balanceada y libre de ruido. Se utiliza como estándar de oro en la academia para ilustrar conceptos de clasificación, regresión y agrupación sin la necesidad de invertir tiempo en la fase de preprocesamiento.

Principales Datasets de Jugete

Iris Flower Dataset (Ronald Fisher, 1936):

Es el dataset de clasificación más famoso de la estadística. Contiene 150 instancias de tres especies de flores Iris (Setosa, Versicolor y Virginica).

- *Variables:* 4 variables continuas (longitud y ancho del sépalo y pétalo) y 1 variable categórica nominal (especie).
- *Uso común:* Ideal para probar algoritmos de clasificación supervisada y visualizaciones de dispersión en el EDA.

Boston Housing Dataset:

Un conjunto de datos clásico para problemas de predicción numérica. Recopila información sobre variables socioeconómicas y geográficas de viviendas en los suburbios de Boston.

- *Variables:* 13 variables cuantitativas (tasa de criminalidad, número de habitaciones, accesibilidad a autopistas) y 1 variable continua objetivo (el valor medio de la vivienda).
- *Uso común:* El estándar introductorio para algoritmos de regresión lineal y regularización.

Titanic Dataset:

Una lista parcial de los pasajeros del famoso naufragio del RMS Titanic. A diferencia de los anteriores, introduce un nivel muy ligero de preprocesamiento debido a la presencia de pocos datos faltantes.

- *Variables:* Mezcla balanceada de variables cuantitativas (edad, tarifa) y cualitativas (género, clase del boleto, puerto de embarque).
- *Uso común:* Clasificación binaria (Sobrevivió / No sobrevivió), ingeniería de características básica y análisis de árboles de decisión.

Wine Dataset:

Resultado de un análisis químico de vinos cultivados en la misma región de Italia pero procedentes de tres cultivares diferentes.

- *Variables:* 13 variables continuas que miden constituyentes químicos (alcohol, ácido málico, ceniza, magnesio, fenoles).
- *Uso común:* Clasificación multiclase y técnicas de reducción de dimensionalidad como PCA (Análisis de Componentes Principales).

Resumen Estructural de los Datasets

La siguiente tabla resume las dimensiones y la tarea analítica principal asociada a cada uno de estos conjuntos de datos estándares en librerías como `scikit-learn` o `R`:

Tabla 2.4: Dimensiones y tareas analíticas de los *toy datasets* más comunes.

Dataset	Instancias (Filas)	Variables (Columnas)	Tipo de Datos	Tarea Predominante
Iris	150	5	Numérico / Categórico	Clasificación Multiclase
Boston Housing	506	14	Numérico	Regresión Numérica
Titanic	891	12	Mixto	Clasificación Binaria
Wine	178	14	Numérico	Clasificación / PCA

2.10.2 Preprocesamiento y Limpieza (*Data Wrangling*)

Frecuentemente, los datos crudos presentan inconsistencias, valores nulos, duplicados o formatos erróneos. Esta etapa matemática y algorítmica se encarga de:

- Imputación o remoción de valores faltantes (*NaN*).
- Normalización y escalado de variables numéricas.
- Codificación de variables categóricas (e.g., *One-Hot Encoding*).

2.10.3 3. Análisis Exploratorio de Datos (EDA)

El EDA (por sus siglas en inglés, *Exploratory Data Analysis*) busca entender la estructura subyacente de las variables mediante estadística descriptiva. Se analizan medidas de tendencia central, dispersión y matrices de correlación para formular hipótesis iniciales e identificar valores atípicos (*outliers*).

2.10.4 4. Modelamiento y Analítica Avanzada

En esta etapa se aplican modelos matemáticos, estadísticos o de aprendizaje automático (*Machine Learning*) según la naturaleza del problema:

$$y = f(X) + \epsilon \tag{2.13}$$

Donde se busca aproximar la función f para tareas de regresión, clasificación o agrupamiento (*clustering*), validando la robustez del modelo mediante métricas de rendimiento específicas.

2.10.5 5. Visualización y Comunicación del Valor

La última etapa traduce los resultados técnicos en insights comprensibles para la toma de decisiones. Implica la construcción de gráficos especializados, reportes automatizados o tableros de control (*dashboards*) que cierran la brecha entre la infraestructura de datos y la estrategia operativa.

2.11 Ecosistema Tecnológico y Perfiles Profesionales

Para comprender el impacto del aprendizaje automático en el mundo real, es fundamental establecer la diferencia entre las tecnologías que procesan la información y los perfiles humanos responsables de diseñar e implementar estas arquitecturas.

2.11.1 1. El Ecosistema Tecnológico (Los Conjuntos)

Desde un punto de vista matemático y conceptual, la Inteligencia Artificial, el *Machine Learning* y el *Deep Learning* no son disciplinas aisladas, sino subconjuntos anidados donde $DL \subset ML \subset IA$.

- **Inteligencia Artificial (IA):** Es el conjunto universal en este contexto. Abarca cualquier técnica, algoritmo o conjunto de reglas heurísticas que permita a una computadora simular el razonamiento humano. Un sistema de reglas lógicas estrictas clasifica como IA.
- **Aprendizaje Automático (ML):** Es un subconjunto de la IA. Prescinde de las reglas explícitas y emplea estadística computacional y optimización matemática para inducir modelos predictivos a partir de datos históricos.
- **Aprendizaje Profundo (DL):** Es un subconjunto estricto del ML. Se restringe al uso de Redes Neuronales Artificiales de múltiples capas ocultas, especializadas en el aprendizaje automático de representaciones sobre datos no estructurados.

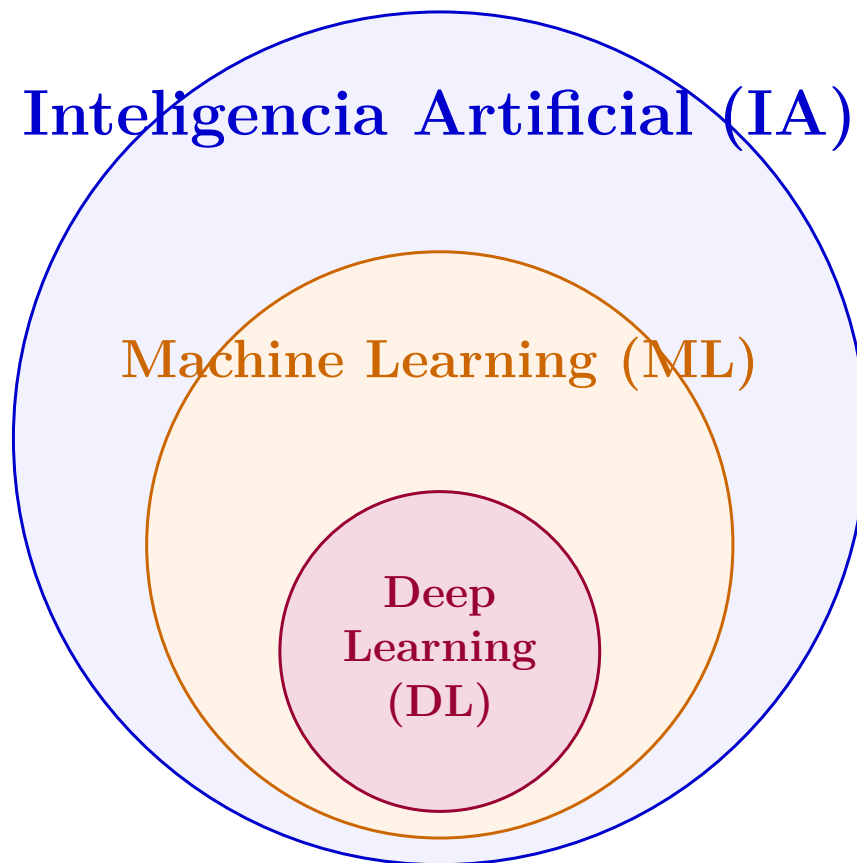


Figura 2.7: Relación de subconjuntos tecnológicos: El Aprendizaje Profundo es una especialización del Machine Learning, que a su vez es una rama de la Inteligencia Artificial.

2.11.2 2. El Ecosistema de Perfiles (La Cadena de Valor del Dato)

La implementación de estas tecnologías en entornos productivos requiere una arquitectura robusta operada por tres perfiles fundamentales, los cuales actúan de forma secuencial:

A. El Ingeniero de Datos (*Data Engineer*):

Es el arquitecto de la infraestructura subyacente. Se encarga del diseño, construcción y mantenimiento de las canalizaciones (*pipelines*) de datos. Utiliza lenguajes como Python y SQL para ejecutar procesos ETL (Extracción, Transformación y Carga). Su trabajo se despliega frecuentemente en plataformas de computación en la nube, orquestando bases de datos, almacenamiento distribuido y servicios de procesamiento sin servidor para asegurar que los datos estén limpios, integrados y altamente disponibles.

B. El Analista de Datos (*Data Analyst*):

Actúa en la capa de diagnóstico. Toma los datos estructurados por el ingeniero y aplica estadística descriptiva para inferir el estado actual y pasado de un fenómeno. Su herramienta principal es la visualización de datos mediante tableros interactivos (*dashboards*) que facilitan la toma de decisiones gerenciales.

C. El Científico de Datos (*Data Scientist*):

Actúa en la capa predictiva y prescriptiva. Su objetivo es modelar la incertidumbre futura aplicando los algoritmos de *Machine Learning* y *Deep Learning* sobre la matriz de datos preparada por el ingeniero. Formula modelos probabilísticos y matemáticos para resolver problemas de regresión, clasificación o agrupamiento.

2.11.3 3. Ejemplo Aplicado: Sistema de Inteligencia Turística

Para ilustrar la sinergia entre estos perfiles, considérese el desarrollo de un Sistema de Inteligencia Turística a escala municipal, cuyo objetivo es gestionar y pronosticar el flujo de visitantes:

1. **Ingeniería de Datos:** El ingeniero desarrolla *scripts* en Python para ingerir datos masivos diarios procedentes de plataformas de alojamiento, sistemas de migración y registros hoteleros. Orquesta estos flujos utilizando infraestructura en la nube (por ejemplo, arquitecturas basadas en *Data Lakes*, catálogos de datos y funciones *Lambda* para procesamiento guiado por eventos), consolidando un repositorio unificado y escalable.
2. **Análisis de Datos:** El analista consulta este repositorio mediante SQL para generar un mapa térmico interactivo que revela la densidad histórica de ocupación por zonas geográficas, respondiendo a la pregunta *¿Cómo se distribuyó el turismo el mes pasado?*.
3. **Ciencia de Datos:** El científico de datos consume la canalización estructurada por el ingeniero y entrena un modelo predictivo, como una red neuronal recurrente o un ensamble de árboles, para proyectar la tasa de ocupación de las próximas cuatro semanas. Esto permite a la administración pública optimizar dinámicamente los recursos logísticos y de movilidad de la ciudad.

PARTE II

Conceptos Fundamentales

PARTE III

Estadística

Bibliografía

- [1] L.B. Castañeda. *Sistema de Aprendizaje Automático*. Colección Texto. Universidad Nacional de Colombia, Facultad de Ciencias, 2004.